

DATA3404 Group Assignment 1

TUT07-AssignmentGroup-06

Individual part

Individual Question 1 : By Zhantao Shi (SID [REDACTED])

This query uses three datasets—listings, reviews, and cities—and joins them. It filters to keep only reviews in Melbourne from the past 365 days. It groups by listing ID and name, uses the aggregate function `COUNT(rs.id)` to count each listing's total reviews, and uses `MAX(rs.review_date)` to get each group's most recent review date. Finally, it orders by review count (highest first), then by listing name alphabetically, and uses `LIMIT 5` to return the top five.

Data Output					Messages	Notifications
	id bigint	listing_name character varying (250)	num_reviews bigint	last_review date		
1	53101911	Modern 1-Bedroom CBD Central Apartment	733	2025-02-17		
2	1039570638701047021	Wanderlust - I want something just like this	127	2025-02-27		
3	35778028	One-Bedroom Apartment	114	2025-03-02		
4	946731014250856459	Romantic East Melbourne Apartment with city vie...	104	2025-02-20		
5	27497868	Yarramunda Bed & Breakfast: Wagyu House	91	2025-02-22		
Total rows: 5 of 5					Query complete 00:00:00.113	

Fig. 1.1.1 Results of queries on a small database(No Index)

Data Output					Messages	Notifications
	id bigint	listing_name character varying (250)	num_reviews bigint	last_review date		
1	53101911	Modern 1-Bedroom CBD Central Apartment	733	2025-02-17		
2	974325482903057482	Melbourne CBD Central Hotel One Bedroom Apartment	175	2025-02-21		
3	42763313	Cozy City Queen Room	140	2025-02-28		
4	42763741	Comfy City Twin Room	132	2025-02-28		
5	1039570638701047021	Wanderlust - I want something just like this	127	2025-02-27		
Total rows: 5 of 5					Query complete 00:00:00.354	

Fig. 1.1.2 Results of queries on a medium database(No Index)

Data Output					Messages	Notifications
	id bigint	listing_name character varying (250)	num_reviews bigint	last_review date		
1	53101911	Modern 1-Bedroom CBD Central Apartment	733	2025-02-17		
2	974325482903057482	Melbourne CBD Central Hotel One Bedroom Apartment	175	2025-02-21		
3	42763313	Cozy City Queen Room	140	2025-02-28		
4	42763741	Comfy City Twin Room	132	2025-02-28		
5	1039570638701047021	Wanderlust - I want something just like this	127	2025-02-27		
Total rows: 5 of 5					Query complete 00:00:00.172	

Fig. 1.1.3 Results of queries on a medium database(With Index)

This is a comparison of query performance without indexing: For the small dataset (Fig. 1.1.1), the query took 113ms; for the medium-sized dataset (Fig. 1.1.2), it took 354ms. Due to the difference in data volume, the query was faster on the smaller dataset. When comparing the query time on the medium-sized dataset before and after indexing: without the index (Fig. 1.1.2), the query took 354ms, while with the index (Fig. 1.1.3), the time was reduced to 172ms. This clearly shows that indexing plays a crucial role in data querying, significantly reducing query time.

Since our query requires performing an equality join on 'listing_id' to quickly locate all reviews associated with a specific listing, and further filtering reviews within the last 365 days, I created a composite index on the 'reviews_medium' table. This index is built on both the 'listing_id' and 'review_date' columns, which helps improve the query performance significantly.

Index Creation

CREATE INDEX idx_reviews ON airbnb.reviews_medium (listing_id, review_date);

Data Output	Messages	Explain	×	Notifications
+ 📄 📁 🗑️ 🔍 📧 🔔 QUERY PLAN text				
5	-> Finalize HashAggregate (cost=24831.12..25192.14 rows=36102 width=58) (actual time=95.037..96.796 rows=7746 loops=1)			
6	Group Key: lm.id, lm.listing_name			
7	Batches: 1 Memory Usage: 2833kB			
8	-> Gather (cost=21371.46..24530.28 rows=30084 width=58) (actual time=90.660..92.052 rows=7746 loops=1)			
9	Workers Planned: 2			
10	Workers Launched: 2			
11	-> Partial HashAggregate (cost=20371.46..20521.88 rows=15042 width=58) (actual time=86.988..87.624 rows=2582 loops=3)			
12	Group Key: lm.id, lm.listing_name			
13	Batches: 1 Memory Usage: 1297kB			
14	Worker 0: Batches: 1 Memory Usage: 1297kB			
15	Worker 1: Batches: 1 Memory Usage: 1297kB			
16	-> Nested Loop (cost=1.59..20221.04 rows=15042 width=58) (actual time=0.194..76.853 rows=35762 loops=3)			
17	-> Hash Join (cost=1.16..8076.29 rows=1875 width=46) (actual time=0.071..13.891 rows=4315 loops=3)			
18	Hash Cond: (lm.city_id = c.id)			
19	-> Parallel Seq Scan on listings_medium lm (cost=0.00..7972.00 rows=22500 width=50) (actual time=0.017..10.780 rows=18000 loops=3)			
20	-> Hash (cost=1.15..1.15 rows=1 width=4) (actual time=0.019..0.020 rows=1 loops=3)			
21	Buckets: 1024 Batches: 1 Memory Usage: 9kB			
22	-> Seq Scan on cities c (cost=0.00..1.15 rows=1 width=4) (actual time=0.013..0.014 rows=1 loops=3)			
23	Filter: ((city_name)::text = 'Melbourne'::text)			
24	Rows Removed by Filter: 11			
25	-> Index Scan using idx_reviews on reviews_medium rm (cost=0.43..6.23 rows=25 width=20) (actual time=0.004..0.013 rows=8 loops=1)			
26	Index Cond: ((listing_id = lm.id) AND (review_date >= (CURRENT_DATE - '365 days'::interval)))			
27	Planning Time: 1.483 ms			
28	Execution Time: 98.303 ms			

According to the Query Execution Plan, the index named 'idx_reviews' was utilized during query execution. Based on the two filter conditions—"listing_id = lm.id" and "review_date >= CURRENT_DATE - INTERVAL '365 days'"—the query can quickly locate the rows that meet the criteria through the index. Compared to a full table scan, an index scan significantly reduces the number of data pages read, thereby improving query execution efficiency.

Individual Question 2: By Yuanfeng Liu (SID [REDACTED])

This query uses the 'hosts' and 'reviews' datasets and performs a join to ensure that only listings with at least one review are included in the statistics. It filters for hosts with an acceptance rate of '100%' and listings with the room type 'Private room'. The results are grouped by host ID and name, and the number of qualifying private room listings for each host is calculated using 'COUNT(DISTINCT ls.id)'. Finally, the output is sorted by 'num_of_listings' in descending order and host name in ascending order, showing the top ten results.

	id integer	host_name character varying (50)	num_of_listings bigint
1	51969157	Jojo	6
2	2413412	Keng Song	6
3	8948251	Joey	5
4	24358640	Janis	4
5	814298	Thatch	4
6	508942455	Alosh	3
7	544927873	Henda	3
8	93130377	Jean	3
9	461577696	Lillian	3
10	12829634	Marcelle & Peter	3

Total rows: 10 of 10 Query complete 00:00:00.156

	id integer	host_name character varying (50)	num_of_listings bigint
1	2413412	Keng Song	52
2	8948251	Joey	29
3	24358640	Janis	18
4	162186374	Wai Yan	16
5	93130377	Jean	15
6	814298	Thatch	14
7	56949831	Sam	13
8	508942455	Alosh	11
9	51969157	Jojo	11
10	30098437	Dragon Hostel	10

Total rows: 10 of 10 Query complete 00:00:00.364

Fig. 1.2.1 Results of queries on a small(left) and medium(right) database(No Index)

	id integer	host_name character varying (50)	num_of_listings bigint
1	2413412	Keng Song	52
2	8948251	Joey	29
3	24358640	Janis	18
4	162186374	Wai Yan	16
5	93130377	Jean	15
6	814298	Thatch	14
7	56949831	Sam	13
8	508942455	Alosh	11
9	51969157	Jojo	11
10	30098437	Dragon Hostel	10

Total rows: 10 of 10 Query complete 00:00:00.318

	id integer	host_name character varying (50)	num_of_listings bigint
1	2413412	Keng Song	52
2	8948251	Joey	29
3	24358640	Janis	18
4	162186374	Wai Yan	16
5	93130377	Jean	15
6	814298	Thatch	14
7	56949831	Sam	13
8	508942455	Alosh	11
9	51969157	Jojo	11
10	30098437	Dragon Hostel	10

Total rows: 10 of 10 Query complete 00:00:00.364

Fig. 1.2.2 Results of queries on a medium database with Index (left) or without Index(right)

Comparison of query performance without indexing: For the small dataset, the query took 156ms; for the medium-sized dataset (Fig. 1.2.1), it took 364ms, indicating faster query execution on the smaller dataset. Comparing the performance before and after indexing on the medium-sized dataset (Fig. 1.2.2): without the index, the query took 364ms, and with the index, it improved slightly to 318ms. Although there was some improvement, the change was not significant.

In this query, we need to filter hosts with an 'acceptance_rate' of '100%' and listings with a room

type of 'Private room'. Therefore, I attempted to create indexes on both 'acceptance_rate' and 'room_type'. However, after comparison, the index on 'room_type' did not effectively speed up the query. As shown in the Query Execution Plan, the index on 'room_type' was not actually utilized during execution. In contrast, the index on 'acceptance_rate' was successfully used and slightly improved the query speed. As a result, I chose to retain only the index on the 'acceptance_rate' column of the 'hosts' dataset.

Index Creation

CREATE INDEX idx_hosts ON airbnb.hosts (acceptance_rate);

Data Output	Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🔍 📥 📈 QUERY PLAN text 🔒		
12	Sort Method: external merge Disk: 2432kB	
13	Worker 0: Sort Method: external merge Disk: 2472kB	
14	Worker 1: Sort Method: external merge Disk: 2528kB	
15	-> Parallel Hash Join (cost=9428.17..92679.17 rows=58009 width=19) (actual time=24.693..227.889 rows=72375 loops=3)	
16	Hash Cond: (rm.listing_id = lm.id)	
17	-> Parallel Seq Scan on reviews_medium rm (cost=0.00..79884.33 rows=833333 width=8) (actual time=0.030..125.162 rows=666667 l...	
18	-> Parallel Hash (cost=9408.60..9408.60 rows=1566 width=19) (actual time=24.515..24.519 rows=1078 loops=3)	
19	Buckets: 4096 Batches: 1 Memory Usage: 256kB	
20	-> Hash Join (cost=1341.11..9408.60 rows=1566 width=19) (actual time=9.052..24.123 rows=1078 loops=3)	
21	Hash Cond: (lm.host_id = h.id)	
22	-> Parallel Seq Scan on listings_medium lm (cost=0.00..8028.25 rows=6288 width=12) (actual time=0.019..13.649 rows=5024 l...	
23	Filter: ((room_type)::text = 'Private room'::text)	
24	Rows Removed by Filter: 12976	
25	-> Hash (cost=1150.72..1150.72 rows=15231 width=11) (actual time=8.906..8.908 rows=15301 loops=3)	
26	Buckets: 16384 Batches: 1 Memory Usage: 776kB	
27	-> Bitmap Heap Scan on hosts h (cost=174.33..1150.72 rows=15231 width=11) (actual time=0.618..5.885 rows=15301 loop...	
28	Recheck Cond: ((acceptance_rate)::text = '100%'::text)	
29	Heap Blocks: exact=786	
30	-> Bitmap Index Scan on idx_hosts (cost=0.00..170.52 rows=15231 width=0) (actual time=0.502..0.502 rows=15301 loo...	
31	Index Cond: ((acceptance_rate)::text = '100%'::text)	
32	Planning Time: 0.364 ms	
33	Execution Time: 333.630 ms	
Total rows: 33 of 33 Query complete 00:00:00.466		

According to the Query Execution Plan, the index named 'idx_hosts' was utilized during query execution to perform lookups based on the filter condition 'acceptance_rate = '100%'.

Group Part

Group Task 1

This query first joins five tables: 'cities', 'neighbourhoods', 'listings', 'hosts', and 'reviews'. It filters the data to include only those entries where the city is 'Sydney', the host is a superhost, the listing price is greater than 100 USD, the review date is between '2024-01-01' and '2025-12-31', and the review comment contains the word 'complaint'. The results are grouped by city and neighbourhood, calculating the number of listings that meet the criteria in each

neighbourhood, as well as the average number of complaints per listing. Finally, the results are sorted in descending order by the number of complaint listings, and the top ten are selected. Below are the query outputs and execution times for the small, medium, and large datasets.

Small Database					Medium Database				
city	nhood_name	num_complaint_listings	avg_complaints_per_listing		city	nhood_name	num_complaint_listings	avg_complaints_per_listing	
character varying (20)	character varying (50)	bigint	numeric		character varying (20)	character varying (50)	bigint	numeric	
1	Sydney	Sydney	8	1.00	1	Sydney	Sydney	29	1.07
2	Sydney	Waverley	4	1.00	2	Sydney	Waverley	20	1.15
3	Sydney	Marrickville	2	1.00	3	Sydney	North Sydney	7	1.00
4	Sydney	North Sydney	2	1.00	4	Sydney	Manly	7	1.00
5	Sydney	Randwick	1	1.00	5	Sydney	Botany Bay	6	1.00
6	Sydney	Auburn	1	1.00	6	Sydney	Parramatta	5	1.00
7	Sydney	Ashfield	1	1.00	7	Sydney	Randwick	4	1.25
8	Sydney	Warringah	1	1.00	8	Sydney	Auburn	2	1.00
Total rows: 8 of 8 Query complete 00:00:00.515					9	Sydney	Hornsby	2	1.00
					10	Sydney	Ku-Ring-Gai	2	1.00
					Total rows: 10 of 10 Query complete 00:00:00.672				

Large Database				
city	nhood_name	num_complaint_listings	avg_complaints_per_listing	
character varying (20)	character varying (50)	bigint	numeric	
1	Sydney	Sydney	49	1.08
2	Sydney	Waverley	27	1.07
3	Sydney	Manly	10	1.00
4	Sydney	North Sydney	9	1.00
5	Sydney	Sutherland Shire	8	1.00
6	Sydney	Randwick	7	1.14
7	Sydney	Botany Bay	5	1.00
8	Sydney	Marrickville	5	1.00
9	Sydney	Warringah	4	1.00
10	Sydney	Parramatta	4	1.00
Total rows: 10 of 10 Query complete 00:00:05.717				

Fig. 2.1.1 Results of queries on a small medium large database Task1 (No Index)

Comparison of query performance without indexing (Fig. 2.1.1): For the small dataset, the query took 515ms; for the medium dataset, 672ms; and for the large dataset, it took 5.717 seconds. As expected, the small dataset, having the least amount of data, had the fastest query time.

This query heavily relies on 'WHERE' clauses for filtering, making it essential to add indexes on the relevant columns. Initially, three indexes were created—each targeting key columns in the 'cities', 'listings', and 'reviews' datasets respectively.

Index Creation

CREATE INDEX idx_city ON airbnb.cities (city_name); [Delete after comparison.]
CREATE INDEX idx_listings ON airbnb.listings small (city_id, price);
CREATE INDEX idx_reviews ON airbnb.reviews small (listing_id, review_date, md5(comments));

According to the Query Execution Plan shown below, the indexes named 'idx_listings' and 'idx_reviews' were used during query execution. The query utilized the 'city_id' index to perform joins between tables, used the 'price' index to filter listings with a price greater than 100, and located reviews from either 2024 or 2025 containing the word 'complaint' in the comment.

However, we found that the 'idx_city' index was not used during the query. This may be because the 'cities' table has a small amount of data, making sequential scanning more efficient. Since unnecessary indexes can increase memory usage and potentially slow down queries, it was removed later. After deletion, keeping only the 'idx_listings' and 'idx_reviews' indexes resulted in better query performance.

Data Output	Messages	Notifications
QUERY PLAN text		
11	-> Hash Join (cost=1736.37..12210.36 rows=107 width=79) (actual time=12.443..363.372 rows=4000 loops=1)	
12	Hash Cond: (ll.host_id = h.id)	
13	-> Nested Loop (cost=0.42..10469.73 rows=436 width=83) (actual time=0.536..367.532 rows=12808 loops=1)	
14	-> Nested Loop (cost=0.00..17.55 rows=46 width=83) (actual time=0.090..0.181 rows=38 loops=1)	
15	Join Filter: (c.id = n.city_id)	
16	Rows Removed by Join Filter: 513	
17	-> Seq Scan on cities c (cost=0.00..1.15 rows=1 width=62) (actual time=0.013..0.015 rows=1 loops=1)	
18	Filter: ((city_name)::text = 'Sydney')::text)	
19	Rows Removed by Filter: 11	
20	-> Seq Scan on neighbourhoods n (cost=0.00..9.51 rows=551 width=21) (actual time=0.007..0.093 rows=551 loops=1)	
21	-> Index Scan using idx_listings on listings_large ll (cost=0.42..227.13 rows=9 width=20) (actual time=0.054..9.629 rows=337 loop...	
22	Index Cond: ((city_id = n.city_id) AND (price > '100')::double precision))	
23	Filter: (n.id = neighbourhood)	
24	Rows Removed by Filter: 12471	
25	-> Hash (cost=1550.41..1550.41 rows=15019 width=4) (actual time=11.882..11.882 rows=15021 loops=1)	
26	Buckets: 16384 Batches: 1 Memory Usage: 657kB	
27	-> Seq Scan on hosts h (cost=0.00..1550.41 rows=15019 width=4) (actual time=0.010..10.283 rows=15021 loops=1)	
28	Filter: (is_superhost = 't')::bpchar)	
29	Rows Removed by Filter: 46132	
30	-> Index Scan using idx_reviews on reviews_large rl (cost=0.56..14.96 rows=1 width=16) (actual time=0.143..0.147 rows=0 loops=4088)	
31	Index Cond: ((listing_id = ll.id) AND (review_date >= '2024-01-01')::date) AND (review_date <= '2025-12-31')::date))	
32	Filter: (lower(comments) ~ '%complaint%')::text)	
33	Rows Removed by Filter: 24	
34	Planning Time: 0.709 ms	
35	Execution Time: 987.876 ms	

The query results after indexing are shown below.

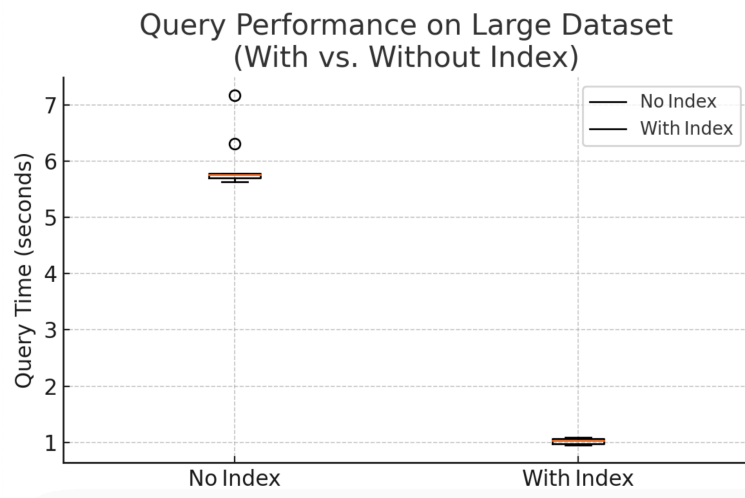
Data Output	Messages	Notifications
	city character varying (20)	nhood_name character varying (50)
1	Sydney	Sydney
2	Sydney	Waverley
3	Sydney	Marrickville
4	Sydney	North Sydney
5	Sydney	Randwick
6	Sydney	Auburn
7	Sydney	Ashfield
8	Sydney	Warringah
Total rows: 8 of 8		Query complete 00:00:00.107
Small database with Index		
Total rows: 10 of 10		
Query complete 00:00:00.348		Medium database with Index
Total rows: 10 of 10		
Query complete 00:00:01.365		Large database with Index

Fig. 2.1.2 Results of queries on a small medium large database Task1 (With Index)

Note: Due to the large size of the table, only the key part showing the execution time is displayed.

Comparison of query performance after indexing (Fig. 2.1.2): For the small dataset, the query took 107ms; for the medium dataset, 348ms; and for the large dataset, 1.365 seconds. These results show that indexing plays a crucial role in improving query performance, significantly

reducing execution time. The improvement is especially noticeable on the large dataset, indicating that the indexes created for this query are appropriate, effective, and impactful.



Based on the large dataset, I ran 10 queries with an index and 10 queries without an index, then used the results to draw this box plot. The plot makes it clear that queries run noticeably faster when an index is present, and the difference between the two cases is easy to see.

Group Task 2

This query is composed of two Common Table Expression(CTE) clauses:

The first CTE joins the listings and reviews tables, calculates the total number of reviews for each listing by city, and computes its percentile ranking within that city. In this part, I used the 'PERCENT_RANK()' function, which ranks listings in descending order of review count within each city and converts their rank into a percentage between 0 and 1. A smaller value indicates a higher number of reviews, meaning the listing is more popular. When extracting the top 20% most-reviewed listings in each city later, we can simply filter for those with a 'popularity_rank' ≤ 0.2 .

The second CTE uses the listings identified in the top 20% and applies an exact match condition using the function '@> ARRAY['Ocean view']::text[]' to filter for those with the 'Ocean view' amenity. These listings are then grouped by city to count how many such popular ocean view listings exist in each one.

Finally, the result is joined with the 'cities' table to output the name of each city along with the total number of popular ocean view listings, sorted alphabetically by city name. The specific output results are shown below.

Small Database		
city_name	num_popular_oceanview_listings	
character varying (20)	bigint	
1	Hong Kong	2
2	Melbourne	10
3	San Francisco	2
4	Sydney	13
5	Vancouver	1
Total rows: 5 of 5 Query complete 00:00:00.214		

Medium Database		
city_name	num_popular_oceanview_listings	
character varying (20)	bigint	
1	Boston	3
2	Brisbane	2
3	Hong Kong	9
4	Melbourne	41
5	San Francisco	23
6	Singapore	1
7	Stockholm	1
8	Sydney	76
9	Vancouver	20
Total rows: 9 of 9 Query complete 00:00:00.621		

Large Database		
city_name	num_popular_oceanview_listings	
character varying (20)	bigint	
1	Boston	4
2	Brisbane	2
3	Hong Kong	20
4	Melbourne	90
5	San Francisco	39
6	Singapore	1
7	Stockholm	3
8	Sydney	158
9	Vancouver	35
Total rows: 9 of 9 Query complete 00:00:01.248		

Fig. 2.2.1 Results of queries on a small medium large database Task2 (No Index)

Comparison of query performance without indexing (Fig. 2.2.1): For the small dataset, the query took 214ms; for the medium dataset, 621ms; and for the large dataset, 1.248 seconds. As expected, the small dataset, with the least amount of data, had the fastest query time.

This query involves filtering on textual data, so I added indexes on the relevant columns. Initially, two indexes were created, targeting key columns in the 'cities' and 'listings' datasets. The difference is that I used a GIN index here, which is specifically designed for full-text search. GIN builds an inverted index by breaking down each element in the array, allowing the query to quickly locate rows containing the specified element by searching the inverted index.

Index Creation

CREATE INDEX idx_listings ON airbnb.listings large USING gin (amenities);

CREATE INDEX idx_cities ON airbnb.cities (city_name);

The figure Below shows a portion of the Query Execution Plan, where it can be seen that the indexes named 'idx_listings' and 'idx_cities' were used during query execution. When joining the popular ocean view listings with the 'cities' table, the B-tree index 'idx_cities' was utilized. Using an Index Scan, the query can directly locate the corresponding city row based on the condition 'city_id = c.id', without scanning the entire table. To filter listings whose 'amenities' include "Ocean view", a Bitmap Index Scan was used. This scan first marks all matching entries

in the index, and then a Bitmap Heap Scan is performed to batch load the corresponding heap pages.

Data Output		Messages	Explain X	Notifications
QUERY PLAN		text		
1	Nested Loop (cost=362238.80..362301.65 rows=12 width=66) (actual time=1238.556..1239.721 rows=9 loops=1)			
2	Join Filter: (pol.city_id = c.id)			
3	Rows Removed by Join Filter: 63			
4	-> Index Scan using idx_cities on cities c (cost=0.14..12.31 rows=12 width=62) (actual time=0.015..0.022 rows=12 loops=1)			
5	-> Materialize (cost=362238.66..362253.83 rows=200 width=12) (actual time=103.207..103.307 rows=6 loops=12)			
6	-> Subquery Scan on pol (cost=362238.66..362252.83 rows=200 width=12) (actual time=1238.483..1239.682 rows=9 loops=1)			
7	-> GroupAggregate (cost=362238.66..362250.83 rows=200 width=12) (actual time=1238.482..1239.680 rows=9 loops=1)			
40	-> Hash (cost=4108.63..4108.63 rows=1356 width=8) (actual time=7.194..7.195 rows=1289 loops=1)			
41	Buckets: 2048 Batches: 1 Memory Usage: 67kB			
42	-> Bitmap Heap Scan on listings_large ll (cost=19.94..4108.63 rows=1356 width=8) (actual time=0.364..7.022 rows=1289 loops=1)			
43	Recheck Cond: (amenities @> '{"Ocean view"}':text[])			
44	Heap Blocks: exact=1231			
45	-> Bitmap Index Scan on idx_listings (cost=0.00..19.60 rows=1356 width=0) (actual time=0.207..0.208 rows=1289 loops=1)			
46	Index Cond: (amenities @> '{"Ocean view"}':text[])			
47	Planning Time: 0.442 ms			
48	Execution Time: 1241.077 ms			

The detailed query results after applying these indexes are shown below.

Data Output

Messages

Explain X

Notifications

city_name

character varying (20)

num_popular_oceanview_listings

bigint

1

Hong Kong

2

2

Melbourne

10

3

San Francisco

2

4

Sydney

13

5

Vancouver

1

Small database with Index

Total rows: 5 of 5

Query complete 00:00:00.154

Data Output

Messages

Explain X

Notifications

≡+

▼

▼

	city_name character varying (20)	num_popular_oceanview_listings bigint
1	Boston	3
2	Brisbane	2
3	Hong Kong	9
4	Melbourne	41
5	San Francisco	23
6	Singapore	1
7	Stockholm	1
8	Sydney	76
9	Vancouver	20

Total rows: 9 of 9

Query complete 00:00:00.529

Medium database with Index

Data Output

Messages

Explain X

Notifications

≡

+

	city_name		num_popular_oceanview_listings	
	character varying (20)		bigint	
1	Boston			4
2	Brisbane			2
3	Hong Kong			20
4	Melbourne			90
5	San Francisco			39
6	Singapore			1
7	Stockholm			3
8	Sydney			158
9	Vancouver			35

Large database with Index

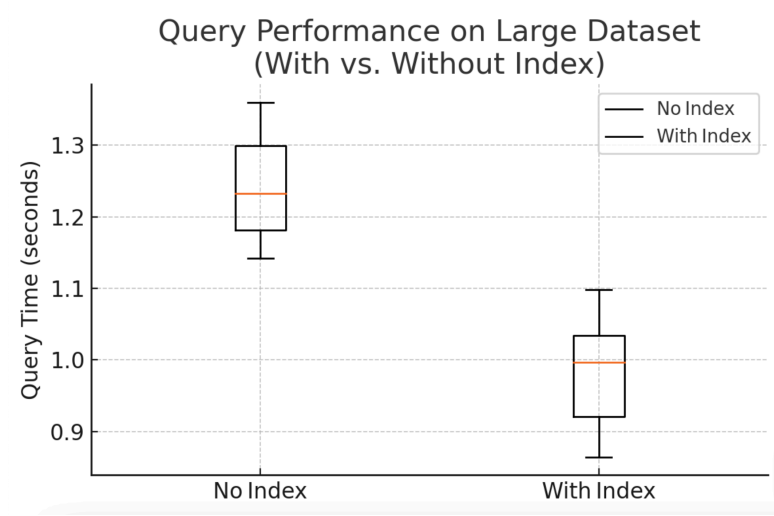
Total rows: 9 of 9

Query complete 00:00:01.049

Fig. 2.2.2 Results of queries on a small medium large database Task2(With Index)

Comparison of query performance after indexing (Fig. 2.2.2): For the small dataset, the query took 154ms; for the medium dataset, 529ms; and for the large dataset, 1.049 seconds. These

results show that the indexes created were successfully utilized during query execution and effectively improved performance, demonstrating noticeable gains in query efficiency.



Similarly, for Group Question 2 I ran 10 query simulations both with and without an index. We can see that the indexed queries run faster, again showing a clear performance improvement.

Group Task 3

I will use the query from Group Task 1 to compare query runtimes between PostgreSQL and Databricks on the large dataset. I found that running the same code on Databricks takes longer(26.91s). PostgreSQL only use 5.717s in same situation. This is mainly because Databricks splits the query into distributed tasks and schedules them across the cluster. The specific explanation of the SQL statement query can be found at the beginning of Group Task 1.

▶ (8) Spark Jobs

Table +

	city	nhood_name	num_complaint_listings	avg_complaints_per_listing
1	Sydney	Sydney	49	1.08
2	Sydney	Waverley	27	1.07
3	Sydney	Manly	10	1.00
4	Sydney	North Sydney	9	1.00
5	Sydney	Sutherland Shire	8	1.00
6	Sydney	Randwick	7	1.14
7	Sydney	Marrickville	5	1.00
8	Sydney	Botany Bay	5	1.00
9	Sydney	Parramatta	4	1.00
10	Sydney	Warringah	4	1.00

10 rows | 26.91s runtime

Fig. 2.3.1 Results of queries on a large database Task3(Databricks)

Data Output					Messages	Notifications
	city character varying (20)	nhood_name character varying (50)	num_complaint_listings bigint	avg_complaints_per_listing numeric		
1	Sydney	Sydney	49	1.08		
2	Sydney	Waverley	27	1.07		
3	Sydney	Manly	10	1.00		
4	Sydney	North Sydney	9	1.00		
5	Sydney	Sutherland Shire	8	1.00		
6	Sydney	Randwick	7	1.14		
7	Sydney	Botany Bay	5	1.00		
8	Sydney	Marrickville	5	1.00		
9	Sydney	Warringah	4	1.00		
10	Sydney	Parramatta	4	1.00		
Total rows: 10 of 10					Query complete 00:00:05.717	

Fig. 2.3.2 Results of queries on a large database Task3(PostgreSQL)

Unfortunately, I tried to add an index in Databricks, but it failed because Databricks does not support creating indexes. Databricks doesn't maintain specialized B-Tree or hash indexes like traditional relational databases; instead, it stores data as Parquet or Delta files in a distributed storage layer. Therefore, it doesn't support CREATE INDEX — its query optimization mainly relies on partitioning and bucketing.

Group contribution statement

	Zhantao Shi	Individual Question 1	Major contributor (yes)
	Yuanfeng Liu	Individual Question 2	Major contributor (yes)